The Android emulator runs behind a virtual firewall that isolates it from your development machine. Because of this, you need to use the special alias `10.0.2.2` (instead of `localhost`, as done with iOS) to loop back to your local Rails server. We'll cover running the app on a physical device in Chapter 9, *Deploy to Physical Devices with TestFlight and Play Testing*.

Now add a single `NavigatorConfiguration` instance wrapped in `listOf()` to convert it to an array, like so:

ch01_22/android/app/src/main/java/com/masilotti/hikingjournal/MainActivity.kt

```
    class MainActivity : HotwireActivity() {
        // ...

»       override fun navigatorConfigurations() = listOf(
»           NavigatorConfiguration(
»               name = "main",
»               startLocation = "$baseURL/hikes",
»               navigatorHostId = R.id.main
»           )
»       )
    }
```

Naming the configuration `"main"` is arbitrary when we only have a single instance. But we'll want to make sure any additional configurations used for tabs are unique.

For the `startLocation` parameter, we append the `/hikes` route to `baseURL`. This will load the homepage when the app launches.

`"$baseURL/hikes"` uses a shorthand version of Kotlin's string interpolation for the value of `baseURL` to build a new URL. In Ruby, this would read `#{baseURL}/hikes`.

`navigatorHostId` needs to reference a node from `activity_main.xml`—Hotwire Native uses this to populate the screen with rendered web content. We named the `FragmentContainerView` node `R.id.main_nav_host` in `activity_main.xml`.